COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	September 11, 2026	**DATE SUBMITTED:**	September 12, 2026
NAME:	CARDENAS, Lea Marie DE VERA, Sophia PANGYARIHAN, Jenny PEÑAFLORIDA, Wrio Carlo PRIETO, Irish Crystle	**INSTRUCTOR:**	Engr. QUEJADO, Jimlord

SEATWORK 8.1 USING SORTING ALGORITHMS 2

1. PROCEDURE OUTPUT

1. Explain the quick sort, shell sort and merge sort types of sorting algorithms.

- **QUICK SORT (def, best and worst time complexity)**
  - ➔ Quick Sort is a divide-and-conquer sorting algorithm. It picks one element from the array called the pivot, then partitions the rest of the array into two groups — elements smaller than the pivot go to the left, elements larger go to the right. It then recursively applies this same process to the left and right sub-arrays, until each sub-array has just one element (which is, by definition, sorted). A real-life way to picture it: imagine sorting a pile of exam papers by score — you pick one paper as a reference ("pivot"), throw all lower scores to the left pile and higher scores to the right pile, then repeat that same trick on each smaller pile. Case Best Case Worst Case Time Complexity O(n log n) O(n²) Program 2: shell_sort_weather.cpp (real-world version) Why The pivot happens to split the array into two roughly equal halves every time, so the array shrinks logarithmically over n total comparisons per level. The pivot happens to be the smallest or largest element every time (e.g., array is already sorted and you always pick the first element as pivot). This creates lopsided partitions — one side has 0 elements, the other has n-1 — so it degenerates into something like nested
  - ➔ **Time Complexity:**
    - **Best Case:** O(n log n), The pivot happens to split the array into two roughly equal halves every time, so the array shrinks logarithmically over n total comparisons per level.
    - **Worst Case:** O(n²), The pivot happens to be the smallest or largest element every time (e.g., array is already sorted and you always pick the first element as pivot). This creates lopsided partitions — one side has 0 elements, the other has n-1 — so it degenerates into something like nested

- **MERGE SORT (def, best and worst time complexity)**
  - ➔ Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

➔ **Time Complexity:**

- **Best Case:** O(n log n), When the array is already sorted or nearly sorted..
- **Worst Case:** O(n log n), When the array is sorted in reverse order.

SHELL SORT (def, best and worst time complexity)

➔ Shell sort is an improvement to insertion sort. It compares elements a certain distance apart, not only adjacent elements. The gap shrinks with each round until it reaches 1, after which the algorithm performs a final insertion sort.

➔ Time Complexity :

- Best Case: O(n log n), Sorts efficiently with less comparisons.
- Worst Case: O(n^2), More comparisons and moves needed.

2. Give real-life examples of the sorting algorithms mentioned.

- **QUICK SORT**
  - ➔ As explained above, the quick sort can be arranged from lowest to highest, or vice versa. Quick sort programs are very useful in different ways and tasks especially when organizing elements and large amounts of data. For example, in system administration and database management, quick sorting techniques can be used to organize files according to their size, prioritize system tasks, or arrange records based on specific values.
  - ➔ Another application of knowledge is in embedded systems, where microcontrollers and sumobots are programmed to identify and process the input signals. Imagine a sumobot where it has to stack blocks according to its weight.

- **MERGE SORT**
  - ➔ Next is the merge sorting technique, where it relies on dividing and merging the parts. We see how merge sort is very useful when tracking online shopping orders/e-commerce. It first sorts data (the items) by filtering it. Like when we're filtering by price, location of the seller, or even alphabetically. It divides the data and merges them back together once sorted into an organized list.
  - ➔ Another example is the CIsco Networking, where it organizes network data, IP addresses, VLAN-ing network devices. Merge Sort techniques can collect a large amount of data then proceed to group them accordingly.

- **SHELL SORT**

➔ This uses the exact same shellSort() function the algorithm doesn't change. What's different is what happens before and after the sort, since it's wrapped around a practical use case.

1. **Input phase** - same idea, but framed as "Day 1 temperature," "Day 2 temperature," etc., collecting 10 daily readings instead of generic numbers.
2. **Sorting** - identical Shell Sort logic runs on the temperature[] array, arranging readings from coldest to hottest.
3. **Extracting real insights from the sorted array** - this is the part that makes it "real world" rather than just a sorting demo:
    - temperature[0] → since the array is now sorted ascending, the very first slot is automatically the coldest reading — no separate search needed.
    - temperature[DAYS - 1] → the last slot is automatically the hottest reading.
    - Median calculation:

      cpp
      C/C++ double median = (temperature[DAYS / 2 - 1] + temperature[DAYS / 2]) / 2.0;

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Grammarly used only to check and correct grammar, spelling, and punctuation errors in the final draft of this work**.** *AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work*

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.